Java 11 Features

Complete Interview Preparation Guide

This document covers all major Java 11 features with:

✓ Simple Explanations
✓ Easy Examples
✓ Interview Questions & Answers
✓ Common Mistakes & Notes
✓ JDK 8 vs JDK 11 Differences

Quick Introduction

Java 11 was released on September 25, 2018, and is a Long-Term Support (LTS) release. It is one of the most important Java versions because it is the first LTS version after Java 8. Many companies migrated from Java 8 directly to Java 11, making it a very common interview topic.

Java 11 brings improvements to String handling, file operations, HTTP communication, garbage collection, and also removes several outdated modules. As a long-term support release, it remains widely used in production environments, making a thorough understanding of its features essential for Java interviews.

Table of Contents

1. 1. New String Methods
2. 2. New File Methods
3. 3. HTTP Client API
4. 4. var in Lambda Parameters
5. 5. Running Single File Java Programs
6. 6. Collection.toArray(IntFunction)
7. 7. Optional.isEmpty()
8. 8. Predicate.not()
9. 9. Nest-Based Access Control
10. 10. Z Garbage Collector (ZGC)
11. 11. Epsilon Garbage Collector
12. 12. Java Flight Recorder & Mission Control
13. 13. Removed & Deprecated Features
14. 14. Migration Guide: JDK 8 to JDK 11

1. New String Methods

Brief Explanation

Java 11 added several new helpful methods to the String class. These methods make common string operations like checking blank strings, splitting into lines, and removing whitespace much easier and cleaner without needing external libraries.

Summary

- isBlank() — checks if string is empty or has only whitespace
- lines() — splits string into a Stream of lines
- strip() — removes whitespace (Unicode-aware, better than trim())
- stripLeading() — removes whitespace only from the beginning
- stripTrailing() — removes whitespace only from the end
- repeat(n) — repeats the string n times

Detailed Description

Before Java 11, handling strings required workarounds: trim() didn't handle all Unicode whitespace, splitting into lines needed split("\n"), and repeating strings needed loops or StringBuilder. Java 11 adds these as built-in methods making the code cleaner.

Simple Examples

isBlank() — Returns true if empty or only whitespace:

```
String s1 = "   ";
System.out.println(s1.isBlank()); // true

String s2 = "Hello";
System.out.println(s2.isBlank()); // false
```

lines() — Splits a multi-line string into a Stream:

```
String text = "Line1\nLine2\nLine3";
text.lines().forEach(System.out::println);
// Output:
// Line1
// Line2
// Line3
```

strip(), stripLeading(), stripTrailing():

```
String s = "  Hello World  ";
System.out.println(s.strip());          // "Hello World"
System.out.println(s.stripLeading());   // "Hello World  "
System.out.println(s.stripTrailing());  // "  Hello World"
```

repeat(n) — Repeats the string:

```
String star = "*";
System.out.println(star.repeat(5)); // *****

String line = "-".repeat(20);
System.out.println(line); // --------------------
```

Interview Questions and Answers

Q1: What is the difference between strip() and trim() in Java 11?

Answer: trim() removes ASCII whitespace (characters with code ≤ 32). strip() is Unicode-aware and removes all Unicode whitespace characters. For example, strip() handles special unicode spaces used in other languages. strip() is generally preferred in Java 11+.

Q2: What does isBlank() return and how is it different from isEmpty()?

Answer: isEmpty() returns true only when the string has zero characters (length == 0). isBlank() returns true when the string is empty OR contains only whitespace characters. Example: ' '.isEmpty() = false, but ' '.isBlank() = true.

Q3: What does lines() return in Java 11?

Answer: lines() returns a Stream<String> where each element is a line from the original string. It splits on line terminators like \n, \r, and \r\n. This is better than split("\n") because it returns a lazy Stream and handles all line endings correctly.

Q4: What happens when repeat(0) is called?

Answer: repeat(0) returns an empty string "". Calling repeat() with a negative number throws an IllegalArgumentException.

Common Mistakes & Important Notes

- ⚠ strip() vs trim(): Always prefer strip() in Java 11+ as it correctly handles Unicode whitespace.
- ⚠ isBlank() vs isEmpty(): A string with spaces is NOT empty but IS blank. Don't confuse them.
- ⚠ lines() returns a Stream — remember to use terminal operations like forEach() or collect().
- ⚠ repeat() throws IllegalArgumentException for negative numbers.

2. New File Methods

Brief Explanation

Java 11 added two very convenient methods to the Files class: readString() and writeString(). These methods make reading from and writing to files much simpler — you can do it in a single line of code instead of writing multiple lines with BufferedReader or FileWriter.

Summary

- Files.readString(Path) — reads entire file content as a String
- Files.writeString(Path, String) — writes a String to a file
- Both methods use UTF-8 charset by default
- Both support optional Charset and OpenOption parameters
- Much simpler than the older BufferedReader / FileWriter approach

Detailed Description

Before Java 11, reading a file as a String required multiple lines of code: creating a BufferedReader, reading line by line, joining with newlines, and closing the stream. Java 11 simplifies this to a single method call. The same applies to writing strings to a file.

Simple Examples

Files.readString() — Read entire file content:

```
import java.nio.file.*;

// Old way (before Java 11)
BufferedReader reader = new BufferedReader(new FileReader("file.txt"));
String content = reader.lines().collect(Collectors.joining("\n"));
reader.close();

// New way (Java 11)
String content = Files.readString(Path.of("file.txt"));
System.out.println(content);
```

Files.writeString() — Write string to a file:

```
// Write to file (creates or overwrites)
Files.writeString(Path.of("output.txt"), "Hello Java 11!");

// Append to existing file
Files.writeString(Path.of("output.txt"), "More text",
                  StandardOpenOption.APPEND);

// Write with specific charset
Files.writeString(Path.of("file.txt"), "Text",
                  StandardCharsets.UTF_16);
```

Interview Questions and Answers

Q1: What are Files.readString() and Files.writeString() methods in Java 11?

Answer: These are two new methods added to java.nio.file.Files class. readString(Path) reads the entire content of a file and returns it as a String. writeString(Path, String) writes a String to a file. Both use UTF-8 charset by default and reduce file I/O code significantly.

Q2: What is the default charset used by readString() and writeString()?

Answer: Both methods use UTF-8 as the default character set. You can specify a different charset by passing it as an additional parameter: Files.readString(path, StandardCharsets.UTF_16).

Q3: What exception do readString() and writeString() throw?

Answer: Both throw IOException (checked exception), so they must be wrapped in try-catch or the calling method must declare 'throws IOException'.

Common Mistakes & Important Notes

- ⚠ These methods throw IOException — always handle with try-catch or declare throws IOException.
- ⚠ readString() loads the ENTIRE file into memory — not suitable for very large files.
- ⚠ writeString() overwrites the file by default — use StandardOpenOption.APPEND to add content.
- ⚠ Path.of() is preferred over Paths.get() in Java 11+ (though both work).

3. HTTP Client API

Brief Explanation

Java 11 officially released a new, modern HTTP Client API in the java.net.http package. It supports HTTP/1.1, HTTP/2, WebSockets, synchronous and asynchronous requests. This replaces the old HttpURLConnection class, which was difficult to use.

Summary

- Package: java.net.http (finalized in Java 11, introduced as incubator in Java 9)
- Main classes: HttpClient, HttpRequest, HttpResponse
- Supports HTTP/1.1 and HTTP/2 automatically
- Supports both synchronous (blocking) and asynchronous (non-blocking) requests
- Supports WebSocket communication
- Much cleaner and easier API than the old HttpURLConnection

Detailed Description

The old HttpURLConnection was hard to use — it required many lines of code just for a simple GET request and had poor HTTP/2 support. The new HttpClient API uses a builder pattern, handles connections efficiently, and provides a clean way to handle responses. It integrates with the CompletableFuture API for async operations.

Key classes: HttpClient (sends requests), HttpRequest (represents a request), HttpResponse (represents a response).

Simple Examples

Synchronous GET request:

```
import java.net.http.*;
import java.net.URI;

HttpClient client = HttpClient.newHttpClient();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/data"))
    .GET()
    .build();

HttpResponse<String> response = client.send(
    request, HttpResponse.BodyHandlers.ofString());

System.out.println("Status: " + response.statusCode());
System.out.println("Body: " + response.body());
```

POST request with JSON body:

```
HttpRequest postRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users"))
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString("{'name':'Alice'}"))
    .build();

HttpResponse<String> res = client.send(
    postRequest, HttpResponse.BodyHandlers.ofString());
System.out.println(res.body());
```

Asynchronous GET request:

```
client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
      .thenApply(HttpResponse::body)
      .thenAccept(System.out::println)
      .join(); // Wait for completion
```

HTTP/2 support:

```
HttpClient http2Client = HttpClient.newBuilder()
    .version(HttpClient.Version.HTTP_2)
    .build();
```

Feature	Old HttpURLConnection	New HttpClient (Java 11)
HTTP/2 Support	No	Yes (automatic upgrade)
WebSocket	No	Yes
Async Requests	No (manual threads)	Yes (CompletableFuture)
Code Simplicity	Complex, verbose	Clean, builder pattern
Package	java.net	java.net.http

Interview Questions and Answers

Q1: What are the three main classes in the new HTTP Client API in Java 11?

Answer: HttpClient (used to send requests), HttpRequest (represents an HTTP request with URI, headers, method, and body), and HttpResponse (holds the status code, headers, and response body).

Q2: How is the Java 11 HTTP Client different from HttpURLConnection?

Answer: The new HttpClient supports HTTP/2 automatically, WebSocket, and asynchronous requests via CompletableFuture. HttpURLConnection only supports HTTP/1.1, requires verbose code, and has no built-in async or WebSocket support.

Q3: How do you make an asynchronous HTTP request in Java 11?

Answer: Use client.sendAsync() instead of client.send(). sendAsync() returns a CompletableFuture<HttpResponse<T>> which you can chain with thenApply(), thenAccept(), etc. for non-blocking processing.

Q4: Does Java 11 HTTP Client support HTTP/2 by default?

Answer: Yes. The HttpClient automatically tries to use HTTP/2 if the server supports it, and falls back to HTTP/1.1 otherwise. You can also explicitly configure the version using .version(HttpClient.Version.HTTP_2) in the builder.

Q5: What is WebSocket support in Java 11 HTTP Client?

Answer: The java.net.http package includes a WebSocket interface for bidirectional communication. You create a WebSocket using HttpClient.newWebSocketBuilder() and implement a WebSocket.Listener to handle messages.

Common Mistakes & Important Notes

⚠ In Java 9 and 10 the HTTP Client was in the 'incubator' module (java.incubator.http). In Java 11 it moved to java.net.http — update your imports.
⚠ HttpClient is immutable and thread-safe — create one instance and reuse it.
⚠ sendAsync() does NOT block — call .join() or .get() if you need to wait for the result.
⚠ Remember to handle IOException and InterruptedException with the synchronous send() method.

4. var in Lambda Parameters (Local-Variable Syntax for Lambdas)

Brief Explanation

Java 11 allows you to use var when declaring parameters in lambda expressions. This was introduced mainly to allow adding annotations to lambda parameters without having to write the full type name. var was introduced in Java 10 for local variables, and Java 11 extends it to lambda parameters.

Summary

- var can now be used in lambda parameter declarations
- Allows adding annotations (@NotNull, @Nonnull, etc.) to lambda parameters
- The type is still inferred by the compiler — var is just syntactic sugar
- All parameters in a lambda must use var, or none at all — you cannot mix
- You still cannot use var with implicit lambda parameters (single parameter without parentheses)

Simple Examples

```
// Java 10 — var for local variable
var list = new ArrayList<String>();

// Java 11 — var in lambda parameter
Consumer<String> printer = (var s) -> System.out.println(s);

// Using var with annotation — the main reason for this feature
Consumer<String> printer = (@NotNull var s) -> System.out.println(s);

// Multiple parameters — ALL must be var
BiFunction<Integer, Integer, Integer> sum = (var a, var b) -> a + b;

// These are all equivalent:
(String s) -> s.length()   // explicit type
(var s)     -> s.length()   // var (Java 11)
s           -> s.length()   // implicit (no parentheses)
```

Interview Questions and Answers

Q1: Why was var introduced for lambda parameters in Java 11?

Answer: The primary reason was to allow annotations on lambda parameters. Without var, you had to write the full type name to add an annotation like @NotNull. With var you can write (@NotNull var s) which is cleaner. The type is still inferred by the compiler.

Q2: Can you mix var and non-var parameters in the same lambda?

Answer: No. Java requires consistency — either all parameters use var, or none use var. Mixing them causes a compilation error. Example: (var a, int b) -> a + b is INVALID.

Q3: Can you use var with a single-parameter lambda without parentheses?

Answer: No. When using var, parentheses are required. So s -> s.length() is valid, but var s -> s.length() is NOT valid. You must write (var s) -> s.length().

Common Mistakes & Important Notes

⚠ Cannot mix: (var a, String b) -> {} is a compile error. Use all var or no var.
⚠ Single parameter without parentheses (s -> ...) cannot use var. You need (var s) -> ...
⚠ The type is still inferred — var does not change behavior, only allows annotations.
⚠ This was added mainly for annotation support, not as a style improvement.

5. Running Single File Java Programs

Brief Explanation

Java 11 allows you to run a single Java source file directly using the java command, without needing to compile it first with javac. This is useful for quick scripts, prototypes, and learning purposes.

Summary

- Run a .java file directly with: java HelloWorld.java
- No need to run javac first — compile and run in one step
- The file is compiled in memory and executed immediately
- Works only for single-file programs (one source file)
- Great for scripting, quick tests, and learning

Simple Examples

Create a file HelloWorld.java:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java 11!");
    }
}
```

Run directly (no separate compile step):

```
# Old way (Java 8)
javac HelloWorld.java
java HelloWorld

# New way (Java 11)
java HelloWorld.java
```

You can also pass arguments:

```
java HelloWorld.java arg1 arg2
```

Interview Questions and Answers

Q1: What is the single-file program feature in Java 11?

Answer: Java 11 allows running a single Java source file directly using the java command without compiling it first. The JVM compiles the source file in memory and executes it immediately. This is useful for quick scripts and learning.

Q2: Does this work for multi-file programs?

Answer: No. This feature only works when all your code is in a single source file. If the program requires multiple .java files, you must compile with javac as before.

Q3: Is the compiled .class file saved to disk?

Answer: No. The compilation happens in memory and the .class file is not saved to disk. Every run recompiles the source file.

Common Mistakes & Important Notes

⚠ Only works for single-file programs — multi-class projects still need javac.
⚠ The .class file is NOT saved to disk — it compiles every time you run.

⚠ The class name does NOT need to match the file name (unlike normal compilation).
⚠ Shebang (#!) lines at the start of the file are supported on Unix/Linux for scripting.

6. Collection.toArray(IntFunction)

Brief Explanation

Java 11 adds a new overload of the toArray() method on Collections. You can now pass an IntFunction (usually an array constructor reference) to get a properly typed array. This replaces the old awkward approach of passing an empty array.

Summary

- New overload: toArray(IntFunction<T[]> generator)
- Returns a properly typed array without casting
- More readable and idiomatic than the old approach
- Works with constructor references like String[]::new

Simple Examples

```
List<String> names = List.of("Alice", "Bob", "Charlie");

// Old way (Java 8) — awkward empty array
String[] arr1 = names.toArray(new String[0]);

// New way (Java 11) — clean constructor reference
String[] arr2 = names.toArray(String[]::new);

System.out.println(Arrays.toString(arr2));
// Output: [Alice, Bob, Charlie]
```

Interview Questions and Answers

Q1: What is the new toArray() method added in Java 11?

Answer: Java 11 adds toArray(IntFunction<T[]> generator) to the Collection interface. You pass an array constructor reference like String[]::new and it returns a typed array. This is cleaner than the old toArray(new String[0]) approach.

Q2: What is the difference between toArray() and toArray(String[]::new)?

Answer: toArray() (no arguments) returns Object[]. toArray(String[]::new) returns String[] — a properly typed array. You need the typed version to avoid ClassCastException when using the array as a specific type.

Common Mistakes & Important Notes

⚠ toArray() without arguments returns Object[] — not a typed array.
⚠ The IntFunction receives the desired array size and must return an array of that size.
⚠ String[]::new is a constructor reference that takes an int (size) and creates a String array.

7. Optional.isEmpty()

Brief Explanation

Java 11 adds the isEmpty() method to the Optional class. This is the opposite of isPresent(). It returns true when the Optional has no value (is empty). This makes code more readable in situations where you want to check if a value is absent.

Summary

- isEmpty() returns true if the Optional contains no value
- It is the logical opposite of isPresent()
- Makes code more readable — avoids !optional.isPresent()
- Available on Optional<T>, OptionalInt, OptionalLong, OptionalDouble

Simple Examples

```
Optional<String> empty = Optional.empty();
Optional<String> value = Optional.of("Java 11");

// Java 8 way — double negation is hard to read
if (!empty.isPresent()) {
    System.out.println("No value"); // prints this
}

// Java 11 way — clean and readable
if (empty.isEmpty()) {
    System.out.println("No value"); // prints this
}

System.out.println(value.isEmpty()); // false
System.out.println(empty.isEmpty()); // true
```

Interview Questions and Answers

Q1: What is Optional.isEmpty() in Java 11?

Answer: isEmpty() is a new method on Optional that returns true if the Optional contains no value (is empty). It is equivalent to !optional.isPresent() and makes the code more readable when checking for the absence of a value.

Q2: What is the difference between isEmpty() and isPresent()?

Answer: isPresent() returns true when a value IS present. isEmpty() returns true when a value is NOT present. They are opposites: isEmpty() == !isPresent().

Common Mistakes & Important Notes

⚠ Do not confuse Optional.isEmpty() with String.isEmpty() or Collection.isEmpty() — they apply to different types.
⚠ isEmpty() and isPresent() always return opposite values for the same Optional.

8. Predicate.not()

Brief Explanation

Java 11 adds Predicate.not() as a static method. It is a negation helper that wraps any predicate and returns its opposite. The main use case is negating method references inside stream filter operations, which was awkward before Java 11.

Summary

- Predicate.not(predicate) returns the logical negation of the given predicate
- Useful for negating method references cleanly
- Replaces the pattern s -> !s.isBlank() with Predicate.not(String::isBlank)
- More readable in stream filter operations

Simple Examples

```
import java.util.function.Predicate;
import java.util.List;
import java.util.stream.Collectors;

List<String> strings = List.of("Alice", "", "  ", "Bob", "Charlie");

// Java 8 way — awkward lambda
List<String> nonBlank1 = strings.stream()
    .filter(s -> !s.isBlank())
    .collect(Collectors.toList());

// Java 11 way — clean method reference negation
List<String> nonBlank2 = strings.stream()
    .filter(Predicate.not(String::isBlank))
    .collect(Collectors.toList());

System.out.println(nonBlank2); // [Alice, Bob, Charlie]
```

Interview Questions and Answers

Q1: What is Predicate.not() in Java 11?

Answer: Predicate.not() is a static factory method that takes a predicate and returns its negation. It is especially useful for negating method references in stream filter operations. For example, Predicate.not(String::isBlank) is equivalent to s -> !s.isBlank().

Q2: Why was Predicate.not() needed if we can write s -> !s.isBlank()?

Answer: Method references cannot be negated directly — you cannot write !String::isBlank. Predicate.not() allows you to negate method references cleanly, which makes code more readable and consistent with the functional style of Java Streams.

Common Mistakes & Important Notes

⚠ Predicate.not() is a static method — call it as Predicate.not(...), not on an instance.
⚠ You can also use Predicate.not() with custom predicates, not just method references.

9. Nest-Based Access Control

Brief Explanation

Java 11 introduces Nest-Based Access Control (JEP 181). Before Java 11, when inner classes accessed private members of their outer class (or vice versa), the compiler generated synthetic bridge methods. Java 11 fixes this by introducing the concept of 'nestmates' — classes that are logically part of the same code unit can directly access each other's private members without synthetic methods.

Summary

- Introduced via JEP 181
- Defines 'nests' — groups of classes that share access (e.g. outer class and its inner classes)
- Eliminates the need for synthetic bridge methods generated by the compiler
- Reflection API updated: getNestHost(), getNestMembers(), isNestmateOf()
- Mostly transparent to developers — it's a JVM-level change

Detailed Description

Before Java 11, when an inner class accessed a private field of its outer class, the Java compiler secretly generated a synthetic package-private method (a 'bridge') to allow this access. This was an implementation workaround that was invisible to developers but could cause issues with security checks and reflection. Java 11 formalizes the concept at the JVM level.

Simple Examples

```
public class Outer {
    private String secret = "hidden";

    class Inner {
        void reveal() {
            // Before Java 11: compiler generates synthetic method
            // Java 11: Inner and Outer are nestmates, direct access
            System.out.println(secret); // Works directly
        }
    }
}

// Reflection API for nests (Java 11)
Class<?> outerClass = Outer.class;
System.out.println(outerClass.getNestHost()); // Outer
System.out.println(Arrays.toString(outerClass.getNestMembers()));
```

Interview Questions and Answers

Q1: What is Nest-Based Access Control in Java 11?

Answer: It is a JVM feature (JEP 181) that allows classes that are logically part of the same code unit (like outer and inner classes) to access each other's private members directly. Before Java 11, the compiler used synthetic bridge methods for this. Java 11 formalizes it at the JVM level.

Q2: What are the new Reflection APIs added for nests?

Answer: Java 11 added getNestHost() (returns the top-level class of the nest), getNestMembers() (returns all classes in the same nest), and isNestmateOf(Class c) (checks if two classes share the same nest) to the Class class.

Q3: Does this change affect everyday coding?

Answer: Mostly no. The change is transparent to developers. The main benefit is eliminating synthetic bridge methods which improves security and reflection behavior. Developers rarely need to use the new getNestHost() APIs directly.

Common Mistakes & Important Notes

⚠ This is a JVM-level change — most developers won't notice it in daily coding.
⚠ The real impact is improved correctness in security managers and reflection-based frameworks.
⚠ Interview questions on this topic typically focus on what it is, why it was needed, and the new reflection APIs.

10. Z Garbage Collector (ZGC)

Brief Explanation

Java 11 introduced the Z Garbage Collector (ZGC) as an experimental feature. ZGC is a low-latency garbage collector designed for applications that need fast response times. It promises pause times of less than 10 milliseconds regardless of heap size (even terabytes of memory).

Summary

- Introduced as experimental in Java 11 (became production-ready in Java 15)
- Designed for low-latency applications with large heaps
- Pause times: typically less than 10ms (most work done concurrently)
- Supports heap sizes from a few megabytes to multi-terabyte
- Uses colored pointers and load barriers for concurrent GC
- Enable with: -XX:+UnlockExperimentalVMOptions -XX:+UseZGC

Detailed Description

Traditional garbage collectors (like G1GC) pause the application (stop-the-world) to do garbage collection, which causes latency spikes. ZGC does almost all its work concurrently while the application is running, resulting in very short pauses. This makes it ideal for real-time and latency-sensitive applications like trading systems, gaming servers, and high-traffic web services.

Simple Examples

Enable ZGC (Java 11 — experimental):

```
# Enable ZGC in Java 11 (experimental)
java -XX:+UnlockExperimentalVMOptions -XX:+UseZGC MyApp

# Set heap size
java -XX:+UnlockExperimentalVMOptions -XX:+UseZGC -Xmx16g MyApp
```

GC	Stop-the-World Pauses	Best For	Java Version
Serial GC	Long	Single-threaded small apps	All
Parallel GC	Medium	Throughput-focused apps	All
G1GC	Short	Balanced latency & throughput	Java 9+ default
ZGC	< 10ms	Low-latency large heaps	Java 11+ (exp), 15+ stable
Epsilon GC	None (no GC)	Performance testing only	Java 11+

Interview Questions and Answers

Q1: What is ZGC and why was it introduced?

Answer: ZGC (Z Garbage Collector) is a low-latency garbage collector introduced as experimental in Java 11. It was designed for applications requiring very short GC pause times (under 10ms) even with very large heaps (multi-terabyte). It performs most GC work concurrently with the application.

Q2: What is the main advantage of ZGC over G1GC?

Answer: ZGC has much shorter stop-the-world pauses — typically under 10ms regardless of heap size. G1GC pauses can grow with heap size. ZGC is better for latency-sensitive applications, while G1GC is a good general-purpose choice.

Q3: Is ZGC production-ready in Java 11?

Answer: No. ZGC was introduced as experimental in Java 11. It became production-ready (non-experimental) in Java 15. In Java 11, you need the -XX:+UnlockExperimentalVMOptions flag to use it.

Common Mistakes & Important Notes

- ⚠ ZGC is experimental in Java 11 — requires -XX:+UnlockExperimentalVMOptions flag.
- ⚠ ZGC became production-ready in Java 15. In interviews about Java 11, mention it as experimental.
- ⚠ ZGC trades some throughput for low latency — not always the best choice for batch processing.
- ⚠ ZGC was originally only available on Linux/x64 in Java 11, with other platforms added later.

11. Epsilon Garbage Collector

Brief Explanation

Java 11 introduces the Epsilon Garbage Collector (JEP 318), also called the 'No-Op' garbage collector. Epsilon allocates memory but deliberately does NOT collect garbage. When the heap is full, the application crashes with OutOfMemoryError. It is NOT meant for production use — it is a testing and performance analysis tool.

Summary

- Introduced in Java 11 as experimental (JEP 318)
- Does NOT collect garbage — it is a no-operation GC
- Application crashes with OutOfMemoryError when heap is full
- Used for: performance testing, memory allocation analysis, short-lived apps
- Enable with: -XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC
- NOT intended for production environments

Simple Examples

```
# Run with Epsilon GC
java -XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC MyApp

# With GC logging to see allocations
java -XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC \
     -verbose:gc MyApp
```

Interview Questions and Answers

Q1: What is the Epsilon Garbage Collector?

Answer: Epsilon GC is a no-operation (no-op) garbage collector introduced in Java 11. It handles memory allocation but never reclaims memory (no garbage collection). When the heap is exhausted, it throws OutOfMemoryError. It is used for performance testing and benchmarking, not production.

Q2: Why would you use a GC that doesn't collect garbage?

Answer: For performance testing: by removing GC overhead completely, you can measure raw application performance and memory allocation rates. It is also useful for short-lived apps that allocate memory but don't live long enough to need collection, and for testing GC-free algorithms.

Common Mistakes & Important Notes

⚠ NEVER use Epsilon GC in production — the application will crash when memory runs out.
⚠ Epsilon GC is experimental in Java 11 — needs -XX:+UnlockExperimentalVMOptions.
⚠ It is useful for benchmarking only when you want to eliminate GC noise from measurements.

12. Java Flight Recorder (JFR) & Java Mission Control (JMC)

Brief Explanation

Java Flight Recorder (JFR) is a low-overhead profiling and diagnostics tool built into the JVM. Java Mission Control (JMC) is a desktop tool to visualize and analyze JFR recordings. Both were previously commercial features of Oracle JDK, but became open-source and free with Java 11 (OpenJDK).

Summary — Java Flight Recorder

- Collects detailed JVM and application performance data
- Very low overhead (typically < 1% performance impact)
- Records: GC events, thread states, memory usage, CPU, exceptions, I/O
- Was a paid Oracle feature before Java 11 — now free and open-source
- Enable with: -XX:+FlightRecorder or jcmd commands

Summary — Java Mission Control

- GUI tool to open and analyze .jfr recording files
- Shows memory, CPU, GC, thread, and I/O visualizations
- Helps identify bottlenecks, memory leaks, thread contention
- Was commercial before Java 11 — now open-source (download separately)

Simple Examples

Start recording with JFR:

```
# Start app with JFR enabled (Java 11)
java -XX:+FlightRecorder \
     -XX:StartFlightRecording=duration=60s,filename=myapp.jfr \
     MyApp

# Using jcmd to start/stop recording on running JVM
jcmd <pid> JFR.start name=MyRecording duration=60s filename=output.jfr
jcmd <pid> JFR.stop name=MyRecording
jcmd <pid> JFR.dump name=MyRecording filename=output.jfr
```

Interview Questions and Answers

Q1: What is Java Flight Recorder (JFR)?

Answer: JFR is a built-in JVM profiling and diagnostics framework. It records detailed information about GC events, threads, memory, CPU, I/O, and exceptions with very low overhead (under 1%). Before Java 11, it was a commercial Oracle JDK feature. In Java 11, it became free and open-source.

Q2: What is Java Mission Control (JMC)?

Answer: JMC is a desktop application for analyzing JFR recording files (.jfr). It provides visual charts and analysis for memory usage, GC behavior, thread activity, and more. It helps developers diagnose performance problems and memory leaks.

Q3: What changed about JFR and JMC in Java 11?

Answer: Both JFR and JMC were commercial (paid) features of Oracle JDK before Java 11. Java 11 open-sourced JFR (now part of OpenJDK), making it free to use. JMC was also open-sourced and is available as a separate download.

Common Mistakes & Important Notes

⚠ JFR is included in the JDK (no separate download needed). JMC must be downloaded separately.
⚠ JFR was a paid feature before Java 11 — using it with Java 8 required an Oracle commercial license.
⚠ JFR has very low overhead and can be used in production environments for continuous monitoring.

13. Removed & Deprecated Features

Brief Explanation

Java 11 removed several Java EE and CORBA modules that were deprecated in Java 9, removed the Java Web Start technology and the Java browser plugin, and deprecated the Nashorn JavaScript engine. Understanding what was removed is important for migration from Java 8.

Java EE Modules Removed

These modules were deprecated in Java 9 and removed in Java 11. They are now available as separate libraries (e.g., Maven/Gradle dependencies).

Removed Module	Contains	Replacement
java.xml.ws	JAX-WS (SOAP Web Services)	jakarta.xml.ws-api (Maven)
java.xml.bind	JAXB (XML Binding)	jakarta.xml.bind-api (Maven)
java.activation	JavaBeans Activation	jakarta.activation (Maven)
java.xml.ws.annotation	Common Annotations	jakarta.annotation-api
java.corba	CORBA support	Third-party ORB
java.transaction	JTA (Transactions)	jakarta.transaction-api
java.se.ee	Aggregator module	Use individual Maven deps

CORBA Removed

CORBA (Common Object Request Broker Architecture) was a legacy technology for distributed object communication. It was deprecated in Java 9 and removed in Java 11. CORBA has been replaced by REST, gRPC, and other modern technologies.

Nashorn JavaScript Engine Deprecated

Nashorn was Java's built-in JavaScript engine (introduced in Java 8). It was deprecated in Java 11 and removed in Java 15. The reason: maintaining a full JavaScript engine that keeps pace with the rapidly evolving JavaScript standard is very costly. GraalVM's JavaScript engine is the recommended alternative.

```
// Nashorn example (Java 8) — deprecated in Java 11
ScriptEngine engine = new ScriptEngineManager().getEngineByName("nashorn");
engine.eval("print('Hello from JavaScript')");

// Warning in Java 11: Nashorn is deprecated, use GraalVM instead
```

Java Web Start & Browser Plugin Removed

Java Web Start (JNLP-based app launcher) and the Java browser plugin (for running applets in browsers) were removed in Java 11. These technologies had become obsolete as modern browsers dropped support for browser plugins, and JNLP was rarely used in new development.

Interview Questions and Answers

Q1: What Java EE modules were removed in Java 11?

Answer: Java 11 removed the following modules that were deprecated in Java 9: java.xml.ws (JAX-WS), java.xml.bind (JAXB), java.activation, java.xml.ws.annotation, java.corba, java.transaction, and java.se.ee. These are now available as separate Maven/Gradle dependencies.

Q2: What is the impact of JAXB removal on existing Java 8 code?

Answer: Code using javax.xml.bind (JAXB) will fail to compile in Java 11. The fix is to add the JAXB API as a Maven/Gradle dependency: jakarta.xml.bind:jakarta.xml.bind-api. This is a very common migration issue when moving from Java 8 to Java 11.

Q3: Why was Nashorn deprecated in Java 11?

Answer: Nashorn was deprecated because maintaining a full JavaScript engine inside the JDK that keeps pace with the fast-evolving JavaScript standard is very expensive. GraalVM's JavaScript implementation is the recommended alternative. Nashorn was removed in Java 15.

Q4: What replaced Java Web Start?

Answer: There is no direct built-in replacement. Alternatives include packaging applications as native installers (using jpackage, available as a preview in Java 14 and stable in Java 16), or using frameworks like Install4j or JNLP alternatives.

Common Mistakes & Important Notes

⚠ Most common migration problem from Java 8: JAXB (javax.xml.bind) is removed — add Maven dependency.
⚠ javax.* packages from Java EE are now jakarta.* (after Eclipse Foundation took over).
⚠ Nashorn is deprecated in Java 11 but not yet removed — removal happened in Java 15.
⚠ Java applets and Java Web Start are gone — there is no backward-compatible replacement.
⚠ CORBA is completely removed — if you use it, you must find a third-party ORB library.

14. Migration Guide: JDK 8 to JDK 11

Brief Explanation

Migrating from Java 8 to Java 11 requires addressing several breaking changes. The most common issues are: removed Java EE modules (especially JAXB), module system restrictions (introduced in Java 9), changed default GC, and removal of the Java browser plugin. This section summarizes what you need to know for interviews.

Most Common Migration Issues

Issue	Java 8 Behavior	Java 11 Behavior	Fix
JAXB (javax.xml.bind)	Built-in	Removed	Add Maven: jakarta.xml.bind-api
JAX-WS (javax.xml.ws)	Built-in	Removed	Add Maven: jakarta.xml.ws-api
sun.misc.Unsafe / internal APIs	Accessible	Warnings/blocked	Use public APIs
Default GC	Parallel GC	G1GC	Test performance; may tune GC flags
javax.activation	Built-in	Removed	Add Maven: jakarta.activation
Nashorn JS Engine	Available	Deprecated (warning)	Switch to GraalVM JS
Java Web Start	Available	Removed	Repackage as native installer
String.chars() streams	Works	Works	No change
-XX:MaxPermSize	Valid JVM flag	Ignored (warning)	Remove from JVM args

Module System (Java 9+) Issues

Java 9 introduced the module system (JPMS). Even if your code is not modularized, the module system affects access to internal JDK APIs. Libraries using sun.misc.* or com.sun.* internal APIs may show warnings or errors.

```
# WARNING in Java 11 for illegal reflective access
WARNING: An illegal reflective access operation has occurred

# Use --add-opens as a temporary workaround
java --add-opens java.base/java.lang=ALL-UNNAMED MyApp

# Or use --illegal-access=permit (removed in Java 17!)
java --illegal-access=permit MyApp
```

Step-by-Step Migration Checklist

- Step 1: Run your app on Java 11 and check for compilation errors
- Step 2: Add Maven/Gradle dependencies for removed Java EE modules (JAXB, JAX-WS, etc.)
- Step 3: Replace or suppress warnings for Nashorn usage
- Step 4: Fix any illegal reflective access warnings (use --add-opens or update libraries)
- Step 5: Remove obsolete JVM flags like -XX:MaxPermSize

- Step 6: Test performance — G1GC is now default (was Parallel GC in Java 8)
- Step 7: Update third-party libraries to versions compatible with Java 11

Interview Questions and Answers

Q1: What is the most common problem when migrating from Java 8 to Java 11?

Answer: The most common issue is the removal of Java EE modules, especially JAXB (javax.xml.bind). Code that used JAXB without explicit dependency will fail to compile. The fix is adding the jakarta.xml.bind-api Maven dependency. Other common issues include illegal reflective access warnings and changed default GC.

Q2: What changed with the default garbage collector from Java 8 to Java 11?

Answer: Java 8 used Parallel GC as the default. Java 9+ changed the default to G1GC (Garbage First). G1GC is better for latency but may have different throughput characteristics. You should test your application's performance after migration.

Q3: What is the module system and how does it affect Java 8 code in Java 11?

Answer: The Java Platform Module System (JPMS) was introduced in Java 9. Even non-modular Java 8 code runs on the classpath (as unnamed module), but access to internal JDK APIs (like sun.misc.*) is restricted. Java 11 shows warnings for illegal reflective access. Libraries using internal APIs must be updated or use --add-opens flags.

Q4: What JVM flags from Java 8 are invalid in Java 11?

Answer: -XX:MaxPermSize is the most common invalid flag (PermGen was removed in Java 8, and the flag was still silently ignored until Java 9 when it started showing warnings). It should be removed from JVM args. Also, -XX:PermSize is invalid.

Q5: Is Java 11 backward compatible with Java 8 code?

Answer: Mostly yes, but not completely. Pure Java 8 code that does not use removed modules, internal APIs, or Nashorn will generally compile and run on Java 11. Code using java.xml.bind, java.xml.ws, or sun.* internal APIs will fail. The source and binary compatibility is high, but removed APIs and module system restrictions require fixes.

Common Mistakes & Important Notes

⚠ javax.xml.bind.* and javax.xml.ws.* will cause 'package not found' errors — add Maven dependencies.
⚠ PermGen is gone since Java 8 — remove -XX:MaxPermSize from all JVM startup scripts.
⚠ --illegal-access=permit was removed in Java 17 — use it only as a short-term fix in Java 11.
⚠ Test with the actual production workload — G1GC may need tuning for your specific use case.
⚠ Update all third-party libraries (Spring, Hibernate, etc.) to versions tested with Java 11.